Report R6 — Code guide and reference

Tier-1 Krylov sector analysis of quantum cellular automata: data structures, algorithms, and file map

QCA Sector Analysis project (Tier 1)

engine_version `1a.1`

Abstract

This report is documentation, not results. It describes the Tier-1 codebase precisely enough to verify every number in R1, R2, and R5 against the source: the exact-arithmetic representation of amplitudes, the construction of the transition graph, the two graph algorithms (union–find for unitary rules, iterative Tarjan plus condensation for dissipative rules), the Tier-1b superoperator and its spectral classification, the fit models and the null-calibrated growth-law extraction, and which Python file is responsible for each of these. Section 2 is the file map; the later sections walk the pipeline stage by stage. Paths are relative to `code/qca_fragmentation/`.

Contents

1 Pipeline at a glance

One rule is a Wolfram number $R \in \{0, \ldots, 255\}$; the analysis is run per (rule, N, bc) *unit*, where N is the chain length and bc $\in \{\texttt{pbc}, \texttt{obc0}\}$ the boundary convention. The stages are:

Tier 1a — graph. Build the directed one-cycle transition graph on the 2^N computational-basis states and decompose it into strongly connected components (SCCs). For unitary rules the SCCs are Krylov sectors; for dissipative rules the terminal SCCs are recurrent classes (attractors) and the rest are transients. Output: one JSON line per unit in `results/`.

Tier 1b — per-class quantum structure. For each recurrent class build the exact restricted superoperator and read its peripheral spectrum to label the attractor *mixing* / *limit cycle* / *coherence-carrying*. Also the exact channel-level diagnostics (graph faithfulness, Cesàro rank).

Tier 1c — scaling. Aggregate the per-N series, fit growth laws with BIC model selection and exact integer recurrences, and emit CSVs, LaTeX tables, and figures.

The graph stage is *exact* — integers only, no floating point and no tolerances (§4). Floating point enters only in Tier 1b, where a dense eigensolve is unavoidable, and in the Tier-1c regressions.

2 Repository / file map

All code lives under `code/qca_fragmentation/`; data and outputs are siblings of `code/` in the repository root.

path	responsibility
`__init__.py`	defines `ENGINE_VERSION = "1a.1"` (stamped into every record; a unit is recomputed only if its stored version differs).
`core/` — rule algebra and one-cycle dynamics	
`core/rules.py`	Wolfram ↔ tuple $(r_{00}, r_{01}, r_{10}, r_{11})$ encoding; `is_unitary`, `channel_kraus_symbols`, reflection / spin-flip maps; HSF oracle numbering.
`core/ring.py`	exact arithmetic in $\mathbb{Z}[1/\sqrt{2}]$: amplitude $(a + b\sqrt{2})/2^m$, addition, $\times \frac{1}{\sqrt{2}}$, exact-zero, norm.
`core/cycle.py`	one brickwork cycle; branch propagation; `succ(x)` (graph edges); `one_cycle_branches_labeled` (Kraus-labelled, for Tier 1b).
`core/ring.py`, `core/cycle.py`	together define the amplitude data structure `Branch` (§5.1).
`graph/` — Tier 1a	
`graph/scc.py`	`analyze` (dispatcher), `sectors_union_find` (unitary), `tarjan` (iterative, dissipative), `_condensation`, `_analyze_condensation`, `recurrent_classes`; the `GraphResult` dataclass.

path	responsibility
quantum/ — Tier 1b	
`quantum/peripheral.py`	`restricted_superoperator`, `classify_attractor` (mixing / limit-cycle / coherent), `geometric_multiplicity`, `graph_faithfulness`, `full_channel_superoperator`, `cesaro_rank`.
`quantum/attractor_structure.py`	the structural coherence test: $\dim_{\text{att}}$ vs. basis content, with a spectral-gap guarantee; `census`.
`quantum/dissipative_report.py`	assembles the R5 census, Tier-1b table, and Cesàro-gap table.
scaling/ — Tier 1c	
`scaling/fits.py`	`fit_series` (M0/M1/M2 BIC), `fit_pure_exponential`, `find_integer_recurrence`, `find_recurrence_by_period`, `name_base`.
`scaling/summary.py`	unitary series extraction + per-rule summary + the `tab_unitary_summary` table.
`scaling/dissipative.py`	dissipative series; `fit_with_period` (period selection); exact bases; `summarize_rule`; CSV + `tab_diss_scaling`.
`scaling/figure.py`	unitary figures incl. the growth-base scatter.
`scaling/diss_figure.py`	dissipative growth-rate scatter maps.
`scaling/growth_map.py`	the unified (base, α) plane and base-vs-base map over all 256 rules.
`scaling/audit.py`	coverage gate: missing / interior-gap / too-short units.
`scaling/overfit_audit.py`	null-model + hold-out calibration of the recurrence and period searches.
drivers and I/O	
`results_io.py`	JSONL record schema, `load_results`, `has_unit`, `record_from_graph_result`, the checkpoint manifest.
`run_rule.py`	analyse one unit (CLI); `--no-ergodic-abort`.
`sweep.py`	idempotent in-process sweep.
`deep_sweep.py`	memory-guarded subprocess sweep for large N.
`tests/`	`pytest` suite (§12).
outputs (repo root)	
`results/{rule}_{bc}.jsonl`	one Tier-1a record per N (append-only).
`checkpoints/manifest.jsonl`	timestamped audit log of completed units.
`analytics/`	census / audit JSON checkpoints.
`figures/`, `reports/`	CSVs, PDFs, and the LaTeX reports.

3 Rule encoding (`core/rules.py`)

An elementary rule $R = \sum_k c_k 2^k$ is decoded per neighbour pair $(m, n) = (x_{i-1}, x_{i+1})$ into a local channel symbol from the output bits $c_p = c_{4m+n}$ (centre 0) and $c_q = c_{4m+n+2}$ (centre 1):

$$(c_p, c_q) = \begin{cases} (0,1) \to \texttt{I} & \text{identity} \\ (1,0) \to \texttt{V} & \text{Hadamard on the centre} \\ (0,0) \to \texttt{D} & \text{reset to } |0\rangle \\ (1,1) \to \texttt{E} & \text{reset to } |1\rangle \end{cases}$$

So each rule is a 4-tuple $(r_{00}, r_{01}, r_{10}, r_{11}) \in \{\texttt{I}, \texttt{V}, \texttt{D}, \texttt{E}\}^4$ (`wolfram_to_tuple` / `tuple_to_wolfram`, a bijection $4^4 = 256$). Key predicates: `is_unitary(t)` = every symbol in $\{\texttt{I}, \texttt{V}\}$ (16 rules); `channel_kraus_symbols(t)` = "has any D/E", i.e. needs a second Kraus branch. The 16 unitary rules carry an HSF oracle number (`wolfram_to_hsf`, obc0, `V` = Hadamard) used only to cross-check against the independent Julia sector counts.

Symmetry maps. `reflect_tuple` swaps $r_{01} \leftrightarrow r_{10}$ (left–right reflection; a valid sector-size reduction for *unitary* rules only, since the even/odd brickwork order matters for dissipative graphs). `spinflip_wolfram` implements $0 \leftrightarrow 1$ conjugation, valid only for `V`-free rules (Hadamard breaks it: $XHX \neq \pm H$).

4 Exact amplitudes: the ring $\mathbb{Z}[1/\sqrt{2}]$ (`core/ring.py`)

Every amplitude produced by an `I/V/D/E` circuit lies in $\mathbb{Z}[1/\sqrt{2}]$. It is stored as

$$\text{value} = \frac{a + b\sqrt{2}}{2^m}, \qquad a, b \in \mathbb{Z} \text{ (arbitrary precision)},$$

with a *single* exponent m shared by a whole branch dictionary. The one non-trivial gate, a conditional Hadamard, multiplies fired components by $1/\sqrt{2}$; with $(a+b\sqrt{2})/\sqrt{2} = (2b+a\sqrt{2})/2$ this stays integer-closed:

operation	effect on numerator (a, b), exponent m
identity / value unchanged, $m \to m+1$	$(a, b) \to (2a, 2b)$ (`double`)
$\times 1/\sqrt{2}$, $m \to m+1$	$(a, b) \to (2b, a)$ (`div_sqrt2`)
exact zero test	$a = 0 \wedge b = 0$ (`is_zero`; no tolerance)
squared norm over 4^m	$\lvert a + b\sqrt{2}\rvert^2 = (a^2 + 2b^2) + (2ab)\sqrt{2}$ (`prob_int`)

Because $\sqrt{2}$ is irrational, $a + b\sqrt{2} = 0$ with integer a, b forces $a = b = 0$: the zero test is exact and the graph never needs a numerical threshold. `renorm` halves all numerators while both parities are even (keeping the integers bounded); `branch_norm` returns the squared norm as two rationals (rational part, $\sqrt{2}$ part) — the $\sqrt{2}$ part must vanish for any physical total, and equals 1 exactly for a normalised unitary branch (a runtime check).

5 One cycle and the transition graph (`core/cycle.py`)

5.1 The `Branch` data structure

$$\texttt{Branch} = (\ \{\, \text{bitmask} \to (a, b) \,\},\ m\ )$$

a dict from computational-basis state (an `int` bitmask, site i = bit i) to its numerator (a, b), plus the shared exponent m. One cycle applies the local update at all *even* sites $0, 2, 4, \ldots$ first, then all *odd* sites (brickwork); odd-layer controls read the post-even-layer state. A rule is *compiled* once per (N, bc) (`_compile`, `lru_cached`) into an ordered list of site steps with precomputed neighbour bit-positions (-1 marks a frozen-0 obc0 edge).

5.2 Kraus branching

A reset (`D/E`) splits a branch into two: the *no-jump* A_0 sub-branch and the *jump* A_1 sub-branch (`_split_site` returns `(a0,a1)`). Branches are **never merged** — merging would fabricate interference between incoherent Kraus alternatives. Two propagators:

- `one_cycle_branches(x,N,t,bc)` — surviving branches after one cycle. Unitary rules take a single-branch fast path (`_apply_site_unitary`).
- `one_cycle_branches_labeled` — additionally tags each branch with the `frozenset` of sites where A_1 fired. That set uniquely names the global Kraus operator $K_b = \prod_i A_{b_i}(i)$ in the fixed processing order, so branches of different inputs can be paired by label to assemble the exact superoperator in Tier 1b.

5.3 The edge relation

$$\texttt{succ}(x) = \{\, y : \ \langle y|\Phi_C(|x\rangle\langle x|)|y\rangle \neq 0 \,\} = \bigcup_{\text{branches}} \text{supp(branch)}, \tag{1}$$

the union of branch supports after one cycle, with exact-zero pruning. This is the directed edge set of the transition graph; it is streamed, never materialised. Equivalently `succ` is the support of the dephased kernel $M_{yx} = \sum_\mu |\langle y|K_\mu|x\rangle|^2$ — exactly the QCA *monitored* in the computational basis after every cycle (see R5 for why this differs from the unmonitored channel for `V`-carrying rules).

6 Tier 1a: graph decomposition (`graph/scc.py`)

`analyze(rule,N,bc,t)` returns a `GraphResult` and dispatches on unitarity.

6.1 Unitary rules: union–find

A unitary (doubly stochastic) channel makes every state recurrent, so each weakly connected component is a single SCC and the sector partition equals the undirected connected components. `sectors_union_find` streams the edges once with union by size and path compression — $O(2^N)$ integers, no stored adjacency, no DFS stack. This is the memory-light path that reaches $N = 22$ (2 GB $\to$ 45 MB at $N = 17$ versus stack-Tarjan). It early-exits to *ergodic* when a component first exceeds $f_{\text{erg}} \cdot 2^N$.

6.2 Dissipative rules: iterative Tarjan + condensation

The directed graph is decomposed by an **iterative** (explicit-stack) Tarjan (`tarjan`) — recursion would overflow at $N \gtrsim 20$. It returns `comp_id` (per-node SCC index, numbered in reverse-topological order) and the component sizes, and early-exits to ergodic if any SCC exceeds $f_{\text{erg}} \cdot 2^N$ or the node count exceeds `node_budget`. Two further streaming passes:

- `_condensation` — builds the successor sets of the condensation DAG (one entry per SCC).
- `_analyze_condensation` — *terminal* SCCs (no outgoing edge) are the recurrent classes; a Kahn topological sort gives the longest path (= `transient_depth`); a reverse-topological DP assigns each transient a single reachable terminal or a "multiple" flag, from which the exclusive `basin` sizes and the `shared_basin_size` follow.

`recurrent_classes` returns the member-state lists of the terminal SCCs and is the entry point Tier 1b consumes.

6.3 The `GraphResult` / record fields

`record_from_graph_result` (`results_io.py`) serialises to one JSON line. The schema (field order in `results_io.FIELDS`):

field	meaning
`rule`, `bc`, `N`	the unit key
`n_scc`	number of SCCs
`n_recurrent`	number of terminal SCCs (sectors / attractors)
`sizes_recurrent`	terminal-SCC sizes, descending (capped at 2048)
`sizes_scc`	all SCC sizes, descending (capped)
`size_hist`	{size:count} histogram (exact reconstruction if capped)
`sizes_truncated`	true if >2048 explicit sizes were dropped
`sizes_basins`	exclusive basin sizes, descending
`shared_basin_size`	states whose forward orbit reaches > 1 terminal
`transient_depth`	longest condensation-DAG path (relaxation proxy)
`n_transient_scc`	number of non-terminal SCCs
`ergodic_flag`, `ergodic_bound`	set when early-exit fired; the SCC size
`attractor_types`, `d_max_quantum`	filled by Tier 1b (else null)
`runtime`, `engine_version`	wall seconds; `"1a.1"`

`sizes_from_record` reconstructs the full multiset from `size_hist` when the explicit list was capped, so the sorted sizes ($D_{\max}$ etc.) are never corrupted by the cap. `load_results` returns $\{N : \text{record}\}$; `has_unit` gates idempotency on `engine_version`.

7 Tier 1b: per-class quantum structure (`quantum/peripheral.py`)

For a recurrent class R (a terminal SCC, closed under the channel) the exact restricted superoperator on $\mathrm{span}(R) \otimes \mathrm{span}(R)^*$ is assembled from the labelled Kraus branches (§5):

$$\Phi_R\big[(y,y'),(x,x')\big] = \sum_b \langle y|K_b|x\rangle\, \overline{\langle y'|K_b|x'\rangle}, \tag{2}$$

a dense $|R|^2 \times |R|^2$ real matrix (`restricted_superoperator`); matrix elements are exact $\mathbb{Z}[1/\sqrt{2}]$ values cast to float only here. Classification (`classify_attractor`) reads the *peripheral* spectrum ($|\lambda| > 1 - 10^{-8}$):

condition	label
unique $\lambda = 1$, no other peripheral eigenvalue	**mixing** (pointer / dark state)
unique $\lambda = 1$, peripheral roots of unity, eigenoperators diagonal	**limit cycle** (period q)
geometric mult. of $\lambda = 1$ exceeds # diagonal fixed operators	**coherence-carrying** ($d_k = \sqrt{\text{mult}}$)

The crucial subtlety is `geometric_multiplicity` (via SVD of $\Phi_R - \lambda I$): the *algebraic* multiplicity is inflated by the transient→recurrent Jordan structure and must not be used. Classes larger than `max_size`$= 32$ are reported *unresolved* (the dense eigensolve is $O(|R|^4)$).

Channel-level diagnostics (validation, $N \lesssim 6$). `full_channel_superoperator` builds the dense $4^N \times 4^N$ channel from per-site Kraus $\{A_0, A_1\}$ as $\prod_{\text{sites}}(A_0 \otimes \bar{A}_0 + A_1 \otimes \bar{A}_1)$. `cesaro_rank` is the geometric multiplicity of its eigenvalue 1 (the protected-coherence dimension). `graph_faithfulness` returns `leak_fix` (Fix(Φ) weight outside the graph's recurrent block) and `leak_flow` (population off the recurrent set after many cycles); a rule is *faithful* iff both vanish.

Structural coherence test (`quantum/attractor_structure.py`). `graph_faithfulness` catches only the weaker failure. The structural test computes $\dim_{\text{att}} = \dim \mathrm{supp}$ of the Cesàro limit of $\Phi^n(\mathbb{1}/d)$ and the number of basis states inside it; the excess `coh_dirs` $= \dim_{\text{att}} -$ `basis_in_att` > 0 is exactly the structure no basis-state graph can represent. It is iterated to a genuine *spectral gap* (smallest kept eigenvalue over largest dropped $>$ `GAP_MIN` $= 10^4$), not a fixed burn-in — burn-in 600 gives rule 19 a bogus $\dim_{\text{att}} = 35$ at $N = 6$; converged it is 5. `census` runs this over all dissipative rules and checkpoints to `analytics/`.

8 Tier 1c: fit models (`scaling/fits.py`)

8.1 BIC model selection: `fit_series`

On the exact $\ln y$ values, three nested models are fitted by least squares and selected by BIC:

$$\text{M0: } \ln y = c; \qquad \text{M1: } \ln y = c + \alpha \ln N; \qquad \text{M2: } \ln y = c + \alpha \ln N + \kappa N.$$

$\text{BIC} = n \ln(\text{RSS}/n) + k \ln n$, with a relative RSS floor so that a near-perfect fit does not let floating-point noise dominate $n \ln \text{RSS}$ (then the fewest-parameter model wins on the $k \ln n$ penalty). The class label: M0 $\to$ *constant*, M1 $\to$ *polynomial*, M2 $\to$ *exponential* iff $|\kappa| >$ `_KAPPA_EPS` $= 0.02$ (a linear sector count yields a spurious $\kappa \sim 0.008$ that must not read as exponential). The $\alpha \ln N$ term is *always* carried when reporting κ, because binomial / charge sectors carry $\sqrt{N}$ prefactors ($\alpha \approx -\frac{1}{2}$) that otherwise bias κ; κ is returned with a leave-one-out spread.

8.2 Pure-exponential rate: `fit_pure_exponential`

The M2 $\alpha \ln N$ term is unstable on the ragged *dissipative* series (rule 90's $D_{\max} = 1, 15, 1, 7, 3, 63, 2, 127$ gives $\alpha = -7.5$ and an impossible base 11). For rate extraction on those, the two-parameter $\ln y = c + \kappa N$ is used instead; it carries a `bounded` flag ($\kappa \le \ln 2$, since $D_{\max} \le 2^N$). The report's rule of thumb (R2): read the base of a *room-packing* (geometric b^N) series from this two-parameter fit, and the base of a *binomial* series from M2 — the mechanism, not the residual, selects the estimator.

8.3 Exact integer recurrences: `find_integer_recurrence`

Searches for the smallest-order linear recurrence $a(n) = \sum_i c_i\, a(n-i)$, $c_i \in \mathbb{Z}$: the first k equations are solved over $\mathbb{Q}$ (`Fraction`), integrality is required, and the candidate is then *verified on every remaining term*. The growth base is $\max |\text{root}|^{1/\text{step}}$ of the characteristic polynomial — an algebraic number, not a regression. Guards: a repeated unit root (a polynomial series) is snapped to base 1 (`np.roots` splits $(x-1)^3$ into a cluster at 1.000007); `step` is the N-spacing (2 for a parity subsequence, so the root is taken to the 1/step power); `max_skip` may step over a finite-size transient but is *off by default* (it costs false positives — §11). Matched constants are named by `name_base` against `_NAMED_BASES` (φ, $\sqrt{3}$, plastic ρ, supergolden, tribonacci, $4^{1/5}$, the parity-doubled surds, ...).

8.4 Period splitting: `find_recurrence_by_period`

Many dissipative series oscillate with N because the attractor is a spatial pattern that only fits a commensurate ring (parity is the common case: an odd `pbc` ring cannot host the two Néel states). `find_recurrence_by_period` runs the recurrence search per residue class $N \bmod p$ and accepts only when *every* class has one and they agree on the base.

9 Tier 1c: scaling drivers

9.1 Unitary — `scaling/summary.py`

`load_series` reads `results/` and builds the per-N series `n_recurrent` (sectors), `d_max` (largest sector), `max_basin`; ergodic-flagged units are excluded and the first ergodic N recorded. `summarize_rule` runs `fit_series` on each and `latex_summary_table` writes `tab_unitary_summary.tex`.

9.2 Dissipative — `scaling/dissipative.py`

The dissipative series are the same four keys but mean attractor count, largest attractor, largest basin, and transient depth. The extra machinery:

- `fit_with_period` chooses a commensurability period by **BIC over the pooled per-class log-residuals**, requiring a decisive margin (`_BIC_MARGIN`= 10, Kass–Raftery "very strong"), at least `_MIN_PER_CLASS`= 4 points per class, and the precondition that the joint fit is actually failing (`_RESID_BAD` = 0.15). An exact per-class recurrence is accepted as an independent second route. This replaced an earlier ratio-of-residuals rule that over-split (§11).
- `summarize_rule` emits, per series, the growth class, base (exact where a recurrence exists), period, and — for *irregular* series — the `irregular_kind`: *arithmetic* (a `V`-free rule is deterministic on basis states, so $D_{\max}$ is a cycle length / multiplicative order, not a growth law — e.g. rule 90 over GF(2)) vs. *undetermined* (a `V`-carrying rule whose period the sizes cannot settle).

`attractor_class` gives the coarse graph structure (unique-point / multistable / extended / multi-extended).

9.3 Figures

`scaling/figure.py` plots $\ln y$ vs N and the unitary growth-*base* scatter; `scaling/diss_figure.py` the dissipative growth-rate maps; `scaling/growth_map.py` the unified maps over all 256 rules — the (base, α) plane (leading rate vs sub-leading power, non-degenerate across exponential / polynomial / ergodic) and the base-vs-base view. A filled marker is an analytically exact base; an open marker is a fit.

10 Sweep drivers and idempotency

`run_rule.run_unit` computes one unit unless it is already cached for the current `engine_version` (`has_unit`); results are append-only, so a sweep is safe to kill and restart. `--no-ergodic-abort` performs the full decomposition even past the ergodic threshold, keeping the flag (used to prove the "ergodic" unitary rules are N-independent; see R2).

`sweep.py` walks $(\text{rule}, N, \text{bc})$ in-process and stops enlarging N once a rule is ergodic or budget-capped (fragmentation cannot reappear at larger N). `deep_sweep.py` runs each unit in a *subprocess* under an address-space `RLIMIT_AS` cap and a wall timeout, in three cost tiers (small / large / huge), so one runaway unit dies alone instead of OOM-ing the night; N is walked upward so an interrupted run still leaves a hole-free dataset at every completed size. The genuine ceiling on 32 GB is $N = 17$ for the dissipative rules ($N = 16$ for the three-Hadamard rules 19/55, whose directed Tarjan does not fit 2^{17} states).

11 Audits and overfitting calibration (`scaling/`)

`audit.py` is the coverage gate: it reports MISSING (no data; expected only for ergodic units), interior GAPS (an absent N inside the range — fatal to a fit, non-zero exit), SHORT (fewer points than a recurrence of a given order needs, $2k{+}1$), and the parity-effective length. `overfit_audit.py` calibrates the two searches that could fake structure:

- **null recurrence rate** — on structureless surrogates the exact recurrence search fires 0/4000; this is what licenses it to certify periods BIC cannot afford.

- **null period rate** — on smooth exponentials +25% noise the calibrated period split fires $\sim 3\%$, versus 21% for the old ratio-of-residuals rule and 33% for plain argmin-BIC.
- **hold-out extrapolation** — refit on all but the largest three sizes and predict them: 854/854 testable series extrapolate exactly, which is far stronger evidence for the algebraic bases than any in-sample residual.

12 Tests (`tests/`)

file	covers
`test_rules.py`	encoding bijection, unitarity, symmetry maps, HSF table
`test_ring.py`	$\mathbb{Z}[1/\sqrt{2}]$ closure, exact zero, norms
`test_cycle.py`	branch propagation, `succ`, unitary single-branch norm
`test_scc.py`	Tarjan / union-find agreement, condensation, ergodic exit
`test_peripheral.py`	superoperator vs dense channel, attractor labels
`test_attractor_structure.py`	rule-27 coherence, gap-driven convergence
`test_fits.py`	recurrences, φ, parity bases, $4^{1/5}$ near-degeneracy
`test_diss_scaling.py`	period selection, null false-split, rule-90 GF(2)
`test_growth_map.py`	anchor-rule $(\text{class}, \text{base}, \alpha)$
`test_deep_sweep.py`	ergodic-skip, cost ordering, tier table

13 Reproducing the pipeline from scratch

All commands use `~/pyenvs/main/bin/python` from `code/`.

1. **One unit:**
   `python -m qca_fragmentation.run_rule --rule 156 --N 6-18 --bc obc0`
2. **Sweep** (idempotent; resume by re-running):
   `python -m qca_fragmentation.sweep --which unitary --n-min 6 --n-max 18`
   `python -m qca_fragmentation.deep_sweep --n-min 14 --n-max 17 --which dissipative`
3. **Coverage gate** (non-zero exit on gaps):
   `python -m qca_fragmentation.scaling.audit`
4. **Scaling tables / CSV:**
   `python -m qca_fragmentation.scaling.summary` (unitary)
   `python -m qca_fragmentation.scaling.dissipative` (dissipative)
5. **Figures:**
   `python -m qca_fragmentation.scaling.figure`
   `python -m qca_fragmentation.scaling.diss_figure`
   `python -m qca_fragmentation.scaling.growth_map`
6. **Overfitting calibration:**
   `python -m qca_fragmentation.scaling.overfit_audit`
7. **Tier 1d — pair graph, certificates, weak charges** (separate `results_tier1d/` store, `tier1d_version 1d.1`; does *not* recompute Tier 1a):
   `python -m qca_fragmentation.run_rule --rule 22 --N 4-12 --bc pbc --tier 1d`
   `python -m qca_fragmentation.pair_sweep --which coherent --n-min 4 --n-max 12`
   `python -m qca_fragmentation.quantum.weak_charges --core`
   `python -m qca_fragmentation.scaling.pair_scaling --bc pbc` (sandwich table)
   `python -m qca_fragmentation.scaling.pair_figure --bc pbc` (coverage / sandwich / parity)

8. **C150 (rule 150) — closed form, flip-graph frontier, spectra** (report R8; `c150.py`, `graph/flip_graph.py`, `quantum/rule150_spectra.py`, `scaling/c150_report.py`; the frontier is checkpointed to `analytics/c150_frontier.jsonl` and is rule-150 only):
   ```
   python -m qca_fragmentation.c150 --verify
   python -m qca_fragmentation.c150 --frontier 21-32 --bc obc0
   python -m qca_fragmentation.scaling.c150_report --quantum
   python -m qca_fragmentation.scaling.c150_report --tables --figures
   ```

9. **Tests:** `python -m pytest qca_fragmentation/tests -q`

Every unit is cached in `results/` and logged in `checkpoints/manifest.jsonl`; nothing recomputes unless `engine_version` changes or `--force` is given.